Real-Time Personalized Sign Language to Speech Conversion Using Media Pipe and Machine Learning

A PROJECT REPORT

Submitted by

KUMUDHAVALLI S	**112822243018**
MONISHA A	**112822243022**
NAVEENA B	**112822243024**
YAMUNA T	**112822243046**

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

T.J.S ENGINEERING COLLEGE,

PERUVOYAL - 601206

(*Affilated to*)

ANNA UNIVERSITY : 600 025

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report "**REAL-TIME PERSONALIZED SIGN LANGUAGE TO SPEECH CONVERSION USING MEDIA PIPE AND MACHINE LEARNING**" is the Bonafide work of "**KUMUDHAVALLI (112822243018), MONISHA A (112822243022), NAVEENA B (112822243024), YAMUNA T (112822243046)**" who carried out the project work under my supervision.

SIGNATURE
Mr.P.SENTHIL, M.Tech
HEAD OF THE DEPARTMENT,
ASSISTANT PROFESSOR,
Department of Artificial Intelligence and Data Science
T.J.S Engineering College,
Peruvoyal-601206

SIGNATURE
Mrs.P.PAVITHRA, M.E.,
GUIDE,
ASSISTANT PROFESSOR,
Department of Artificial Intelligence and Data Science
T.J.S Engineering College,
Peruvoyal-601206

CERTIFICATE OF EVALUATION

COLLEGE NAME	: T.J.S ENGINEERING COLLEGE
DEGREE/BRANCH	: B.TECH / ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
SEMESTER	: VIII
NAME OF STUDENTS	: KUMUDHAVALLI S (112822243018) MONISHA A (112822243022) NAVEENA B (112822243024) YAMUNA T (112822243046)
TITLE OF THE PROJECT	: REAL TIME PERSONALIZED SIGN LANGUAGE TO SPEECH CONVERSION USING MEDIAPIPE AND MACHINE LEARNING
SUPERVISOR	**:** Mr.P.SENTHIL, M.Tech. ASSISTANTPROFESSOR

The report of this project has been submitted by the above student in partial fulfillment for the award of the degree **Bachelor of Artificial Intelligence and Data Science** from Anna University Chennai. The report has been evaluated and the authority of the work has been confirmed.

Submitted for the University Examination held on ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to express the gratitude and thanks to our beloved and most respected **Chairman "Kalvineri Kavalar" Shri. T.J. Govindarajan B.A., MLA.,** for his support.

I would like to express the gratitude to our **Principal Dr. J. Prakash, M.E., Ph.D.,** who as an inspiration for us to perform & raise our institution name and recognition.

I wish to express our sincere thanks and gratitude to **Mr. P. Senthil M.Tech., Head of the Department of Artificial Intelligence and Data Science,** who has been guiding force and constant source of inspiration to us.

With respect we express gratitude to our Guide **Mrs. P. Pavithra, M.E.,** Assistant professor, Department of Artificial Intelligence And Data Science, for having extended his co-operation and guidance in doing this project.

I express our gratitude to our **Parents and Friends** for their encouragement and support. We will be failing in our duty if we don't acknowledge the help and support given by all these people who have directly or indirectly helped us in completing our project.

ABSTRACT

This project aims to develop an advanced Sign-to-Speech system designed to facilitate seamless communication for speech-impaired individuals by converting personalized sign language gestures into spoken sentences. Utilizing Media-Pipe's state-of-the-art hand tracking and gesture recognition capabilities, combined with robust machine learning algorithms, the system achieves precise and efficient recognition of dynamic hand gestures in real-time.

Unlike conventional systems that focus primarily on isolated single-word signs or require controlled backgrounds, our proposed method operates effectively in any environment and supports personalized sentence-level sign recognition such as "I need water," "I am hungry," "Hello," and "How are you," thereby enabling users to convey complex messages. The system is implemented using Python and OpenCV, enhancing its accessibility and integration potential with existing platforms.

Through this innovative approach, the project aspires to empower speech-impaired individuals with an intuitive and practical communication tool, improving their social interactions and overall quality of life.

Keywords: Sign language recognition, Media-Pipe, machine learning, hand gesture tracking, personalized sign sentences, OpenCV, speech impairment communication, real-time sign-to-speech system.

தமிழ் சுருக்கம்

இந்த திட்டத்தின் முக்கிய நோக்கம் பேச்சு குறைபாடு கொண்ட நபர்களுக்கு தொடர்பு கொள்ள உதவும் மேம்பட்ட Sign-to-Speech அமைப்பை உருவாக்குவது ஆகும். இந்த அமைப்பு தனிப்பயன் கைகுறி (Sign Language) அசைவுகளை கண்டறிந்து அவற்றை பேசும் வாக்கியங்களாக மாற்றுகிறது. இதற்காக Media-Pipe வழங்கும் மேம்பட்ட கை கண்காணிப்பு (Hand Tracking) மற்றும் கைகுறி அடையாளம் காணும் தொழில்நுட்பங்களுடன் Machine Learning அல்காரிதங்கள் இணைக்கப்படுகின்றன. இதன் மூலம் நேரடி நேரத்தில் (Real-time) கை அசைவுகளை துல்லியமாகவும் வேகமாகவும் அடையாளம் காண முடிகிறது.

பொதுவாக இருக்கும் அமைப்புகள் தனிப்பட்ட ஒரு வார்த்தை கைகுறிகளை மட்டுமே அடையாளம் காணும் அல்லது குறிப்பிட்ட சூழலில் மட்டும் செயல்படும். ஆனால் இந்த திட்டத்தில் முன்மொழியப்பட்ட அமைப்பு எந்த சூழலிலும் செயல்படக்கூடியதாகவும், “I need water”, “I am hungry”, “Hello”, “How are you” போன்ற முழு வாக்கியங்களைக் கொண்ட தனிப்பயன் கைகுறிகளையும் அடையாளம் காணக்கூடியதாகவும் வடிவமைக்கப்பட்டுள்ளது. இதன் மூலம் பயனர்கள் தங்கள் எண்ணங்களை முழுமையாக வெளிப்படுத்த முடியும்.

இந்த அமைப்பு Python மற்றும் OpenCV பயன்படுத்தி உருவாக்கப்பட்டுள்ளது. இது பயன்பாட்டிற்கு எளிதாகவும், பிற அமைப்புகளுடன் ஒருங்கிணைக்கவும் ஏற்றதாகவும் உள்ளது.

இந்த புதிய அணுகுமுறையின் மூலம் பேச்சு குறைபாடு கொண்ட நபர்கள் எளிதாகவும் பயனுள்ளதாகவும் தொடர்பு கொள்ள உதவும் ஒரு செயல்முறை கருவியை வழங்குவது இந்த திட்டத்தின் நோக்கமாகும். இது அவர்களின் சமூக தொடர்புகளையும் வாழ்க்கைத் தரத்தையும் மேம்படுத்த உதவும்.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	**iv**
	LIST OF FIGURES	**xi**
	LIST OF TABLES	**xii**
	LIST OF ABBREVIATIONS	**xiii**
1	**INTRODUCTION**	**1**
	1.1 Introduction	**1**
	1.2 Background of the Study	**2**
	1.3 Problem Statement	**2**
	1.4 Objectives	**2**
	1.5 Scope of the Project	**3**
	1.6 Organization of the Report	**4**
2	**LITERATURE SURVEY**	**5**
	2.1 Overview of Sign Language Recognition Systems	**5**
	2.2 Vision-Based Hand Gesture Recognition	**5**
	2.3 Hand Landmark Detection Using Media-Pipe	**5**
	2.4 Speech Output Integration	**6**
	2.5 Limitations of Existing Systems	**6**
	2.6 Real-Time Processing in Sign Language Systems	**6**

CHAPTER	TITLE	PAGE NO
3	**SYSTEM ANALYSIS**	**7**
	3.1 Existing System	**7**
	3.2 Proposed System	**8**
	3.3 Advantages of the Proposed System	**9**
	3.4 Feasibility Study	**10**
	3.4.1 Technical Feasibility	**10**
	3.4.2 Economic Feasibility	**10**
	3.4.3 Operational Feasibility	**10**
4	**SYSTEM DESIGN**	**11**
	4.1 System Architecture	**11**
	4.2 Data Flow Diagram	**12**
	4.2.1 DFD Level 0 (Context Diagram)	**12**
	4.2.2 DFD Level 1	**12**
	4.2.3 Data Stores and Data Flow	**13**
	4.3 Use Case Diagram	**15**
	4.4 Module Description	**15**
	4.5 Sign-to-Speech Data Flow Diagram	**16**
5	**IMPLEMENTATION**	**17**
	5.1 Tools and Technologies Used	**17**
	5.2 Dataset Preparation	**17**
	5.3 Model Training	**24**

CHAPTER	TITLE	PAGE NO
	5.4 Software Requirements	**24**
	5.5 Hardware Requirements	**25**
	5.6 System Integration	**25**
6	**TRAINING OF RANDOM FOREST CLASSIFIER**	**26**
	6.1 Overview	**26**
	6.2 Data Distribution and Splitting	**27**
	6.3 Training and Validation Process	**28**
	6.4 Evaluation on Unseen Test Data	**29**
	6.4.1 Classification Report	**29**
	6.4.2 Confusion Matrix	**30**
7	**RESULTS AND DISCUSSION**	**72**
	7.1 Model Performance Results	**72**
	7.2 Live Hand Capture	**72**
	7.3 Hand Tracking Detection	**73**
	7.4 Sign to Speech Output	**74**
	7.5 Discussion	**75**
8	**CONCLUSION AND FUTURE WORK**	**76**
	8.1 Conclusion	**76**
	8.2 Future Enhancements	**77**

CHAPTER	TITLE	PAGE NO
	APPENDIX – I	
	Collect Image.py	**31**
	Create DataSet.py	**42**
	Train Classifier.py	**49**
	Sign Classifier.py	**60**
	Software and Libraries	**79**
	APPENDIX – II	
	REFERENCES	**80**
	APPENDIX – III	**82**

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
4.1	System Architecture Diagram	11
4.2.3	Data Flow Diagram	14
4.5	Sign-to-Speech Data Flow Diagram	16
5.2.1	Dataset-“How are you”Sign	18
5.2.2	Dataset-“I need water”Sign	19
5.2.3	Dataset-“Hello”Sign	20
5.2.4	Dataset-“I need help”Sign	21
5.2.5	Dataset-“I am hungry”Sign	22
5.2.6	Dataset-“No”Sign	23
7.2	Live Hand Capture	72
7.3	Hand Tracking Using Media-Pipe	73
7.4	Sign to Speech Convertor Output	74

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
6.1	Random Forest Model Specifications	26
6.2	Dataset Class Distribution	27
6.3	Summary of Model Performance Metrics	28
6.4.1	Classification Performance across All Classes	29
6.4.2	Heatmap Representation of Confusion Matrix	30

LIST OF ABBREVIATIONS

S. NO	ABBREVIATIONS	FULL FORM
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	CNN	Convolutional Neural Network
4	LSTM	Long Short-Term Memory
5	ANN	Artificial Neural Network
6	ISL	Indian Sign Language
7	DFD	Data Flow Diagram
8	GUI	Graphical User Interface
9	TTS	Text-to-Speech
10	GTTS	Google Text-to-Speech
11	FPS	Frames Per Second
12	API	Application Programming Interface
13	RGB	Red Green Blue
14	IDE	Integrated Development Environment
15	CPU	Central Processing Unit

CHAPTER 1

INTRODUCTION

1.1 Introduction

Communication is fundamental to human interaction, yet speech impairment can create significant barriers that affect social integration, education, and overall quality of life. Sign language offers a powerful alternative for many speech-impaired individuals, but widespread understanding and interpretation by the general population remain limited. This gap highlights the need for effective automated systems that can translate sign language into spoken or written language to promote inclusivity.

Recent advances in computer vision and machine learning have paved the way for more accurate and real-time sign language recognition systems. Among these, Media-Pipe provides a high-fidelity hand and finger tracking framework that captures intricate hand gestures, enabling detailed analysis and interpretation of signs in various environments. By integrating Media-Pipe with machine learning algorithms, it is possible to develop robust systems capable of recognizing not just isolated signs but also complete personalized sentences.

Unlike many existing solutions that recognize single words or require controlled backgrounds, our proposed method aims to support dynamic and personalized sentence-level sign language recognition in any setting. This includes common expressions such as "I need water," "I am hungry," "Hello," and "How are you," which are essential for everyday communication. Utilizing OpenCV and Python ensures that the system remains accessible, flexible, and easy to deploy across different platforms.

This project aspires to empower speech-impaired individuals by providing a natural and efficient communication tool that bridges the gap between sign

language users and the broader community, enhancing social interaction, independence, and overall well-being.

1.2 Background of the Study

Sign language recognition has gained significant attention in recent years due to the increasing need for assistive technologies. Traditional systems focused mainly on recognizing individual alphabets or static gestures. However, real-world communication requires dynamic and sentence-level gesture recognition.

Modern frameworks like Media-Pipe enable accurate real-time hand landmark detection. These advancements make it possible to extract hand features efficiently and perform gesture classification using machine learning models.

Despite technological improvements, challenges such as lighting variation, background noise, and real-time processing constraints still exist. Therefore, there is a need for an efficient, lightweight, and accurate system that works in real-world conditions.

1.3 Problem Statement

People with hearing and speech impairments rely on sign language for communication. However, most people are not familiar with sign language, leading to communication barriers. Existing systems often recognize only single letters or require controlled environments.

There is a need to develop a real-time system that can accurately recognize personalized sign gestures and convert them into speech output, enabling effective communication in everyday situations.

1.4 Objectives

1. To design and implement a real-time sign language recognition system using Media-Pipe for accurate hand and gesture tracking.

2. To develop machine learning models capable of recognizing personalized sign language sentences rather than isolated words or alphabets.
3. To ensure the system can function effectively across varied and unconstrained background environments without requiring a plain backdrop.
4. To convert recognized sign language sentences into clear and natural speech output using Python and OpenCV.
5. To validate the system's performance in terms of accuracy, speed, and usability in real-world conditions.
6. To provide a cost-effective communication aid that can enhance the independence and social inclusion of speech-impaired individuals.

1.5 Scope of the Project

The project focuses on developing a real-time continuous sign language detection system that accurately recognizes personalized sign sentences using standard camera input. It employs Media-Pipe for efficient hand and finger tracking alongside machine learning algorithms to interpret dynamic hand gestures in diverse background environments without restrictions.

The system converts recognized personalized sign sequences directly into spoken language, facilitating effective communication for speech-impaired individuals. Implementation utilizes Python and OpenCV for image processing and integrates speech synthesis for clear audio output.

This approach supports customized, user-specific sign phrases and enables natural, fluid communication by processing continuous gestures rather than isolated signs. The project aims to provide a practical, accessible tool for enhancing daily interactions and social inclusion of speech-impaired users.

1.6 Organization of the Report

Chapter 1 – Introduction

This chapter introduces the project, explains the background of the study, problem statement, objectives, and scope of the project.

Chapter 2 – Literature Review

This chapter reviews existing research works, technologies, and methods related to sign language recognition and speech conversion systems.

Chapter 3 – System Analysis

This chapter discusses the existing system, its limitations, and the proposed system along with feasibility analysis.

Chapter 4 – System Design

This chapter explains the overall architecture, data flow diagrams, use case diagrams, and module descriptions of the system.

Chapter 5 – Implementation

This chapter describes the tools and technologies used, dataset preparation, model training, and system development process.

Chapter 6 – Training of Random Forest Classifier

This chapter explains the training process of the Random Forest Classifier, including dataset preparation, data splitting, and model evaluation using classification report and confusion matrix.

Chapter 7 – Results and Discussion

This chapter presents the system results, including model performance, live hand capture, hand tracking detection, and sign-to-speech output.

Chapter 8 – Conclusion and Future Work

This chapter summarizes the project outcomes and suggests possible future improvements for the system.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of Sign Language Recognition Systems

Sign language recognition systems are developed to help hearing and speech-impaired individuals communicate effectively with others. These systems detect hand gestures and convert them into readable text or speech output. Traditional communication methods relied on human interpreters, which are not always available and may be expensive.

Modern sign language systems use computer vision techniques to capture hand gestures through a webcam and process them in real time. These systems aim to provide faster and more accessible communication in public places, educational institutions, and workplaces.

2.2 Vision-Based Hand Gesture Recognition

Vision-based gesture recognition systems use a camera to capture hand movements. The captured video frames are processed using image processing techniques to detect and track the hand region. These systems analyse hand shape, finger position, and movement patterns to identify specific signs.

Compared to sensor-based systems that use gloves and hardware devices, vision-based systems are more convenient and cost-effective because they do not require wearable equipment.

2.3 Hand Landmark Detection Using Media-Pipe

Recent systems use Media-Pipe for real-time hand tracking. Media-Pipe detects 21 key landmarks on each hand, including fingertips, joints, and wrist position. The coordinates of these landmarks are used to identify different hand gestures. Landmark-based detection reduces the need for complex image preprocessing

and improves real-time performance. However, performance may vary depending on lighting conditions and camera quality.

2.4 Speech Output Integration

After recognizing the hand gesture, the system converts the identified text into speech using Text-to-Speech (TTS) technology. This allows the recognized sign to be spoken aloud, enabling communication with normal individuals.

Speech integration improves the practical usability of sign language systems and enhances real-time interaction between users.

2.5 Limitations of Existing Systems

Existing sign language recognition systems have certain limitations:

- Sensitivity to lighting and background variations
- Difficulty in recognizing continuous gestures
- Limited number of supported gestures
- Dependence on proper hand positioning
- Reduced accuracy when hands overlap

These limitations show the need for an improved real-time sign language to speech conversion system that is simple, efficient, and user-friendly.

2.6 Real-Time Processing in Sign Language Systems

Real-time processing is essential for effective communication. The system must capture video frames, detect hand landmarks, recognize gestures, and generate speech output without noticeable delay.

Efficient frame processing and optimized code help maintain smooth performance. Real-time systems improve user experience and make the communication process more natural and interactive.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

Many existing sign language recognition systems primarily focus on recognizing isolated single alphabet signs or individual words. These systems typically require users to perform gestures against a plain white or uniform background to achieve accurate detection and classification. Such environmental constraints limit their usability in real-world situations where backgrounds are varied and uncontrolled. Consequently, the user experience can be restrictive and less natural, reducing the effectiveness of communication for speech-impaired individuals.

Additionally, some current approaches attempt to convert speech into sign language animations or gestures (speech-to-sign). However, this approach can be illogical and less useful for the target users. Since speech-impaired individuals rely on sign language as their primary communication mode, simply converting speech to sign animations does not facilitate communication with people unfamiliar with sign language. The opposite person cannot understand these signs unless they know sign language, thereby not addressing the vital need for translating signed gestures into understandable speech or text.

Furthermore, many existing systems do not support sentence-level or personalized signs but instead focus on isolated words or fixed vocabularies. This limitation makes it difficult for users to express complex thoughts or everyday phrases naturally. Moreover, a majority of the current technologies require significant computational resources and extensive training datasets, impacting their real-time performance and accessibility.

In contrast, our proposed system utilizes Media-Pipe's high-precision hand tracking to recognize dynamic, personalized sign sentences without the need for

controlled backgrounds or restrictive environments. By integrating machine learning algorithms, our solution aims to accurately translate full sign language sentences into speech in real time, thereby overcoming the constraints of existing systems and promoting more natural and effective communication for speech-impaired individuals.

3.2 Proposed System

The proposed Sign-to-Speech system aims to overcome the limitations of existing methods by providing an efficient, real-time solution for translating personalized sign language sentences into spoken words, thereby enhancing communication for speech-impaired individuals. This system leverages Media-Pipe's advanced hand tracking technology, enabling accurate recognition of complex hand gestures in diverse and unconstrained backgrounds without the need for a controlled environment.

Unlike traditional systems that recognize only isolated signs or require uniform backgrounds, our approach supports dynamic, sentence-level sign recognition, allowing users to communicate full phrases such as "I need water," "I am hungry," and "How are you." This personalized recognition capability accommodates variations in signing style and vocabulary, making the system more adaptable and user-friendly.

By integrating robust machine learning algorithms, the system efficiently processes gesture data captured through a standard camera and interprets the signs into corresponding speech output using OpenCV and Python. The use of widely accessible hardware and software tools ensures affordability and ease of deployment.

Overall, the proposed system provides a more natural, flexible, and practical communication tool that operates effectively across varying environments and user preferences. It empowers speech-impaired individuals by bridging the

communication gap with others, facilitating smoother social interactions and improving their quality of life.

3.3 Advantages of the Proposed System

1. **Sentence-Level Recognition:** Unlike many existing systems limited to single words or alphabets, this system recognizes complete personalized sentences, enabling more natural and meaningful communication.
2. **Robust to Various Backgrounds:** Utilizes Media-Pipe's advanced hand-tracking, allowing the system to work accurately in any background environment without the need for a plain or white backdrop.
3. **Real-Time Processing:** Efficient use of machine learning and OpenCV ensures quick gesture recognition and speech generation, facilitating smooth, real-time interactions.
4. **Personalization:** Supports customized signs and varied signing styles, making it adaptable to individual user needs and preferences.
5. **Accessibility and Affordability:** Implemented in Python using commonly available hardware (standard cameras), the system is cost-effective and easy to deploy.
6. **Improved Communication for Speech-Impaired Users:** Bridges the communication gap by converting sign language into understandable speech, aiding interaction with those unfamiliar with sign language.
7. **Enhanced User Experience:** Supports dynamic gestures and continuous sentences rather than static, isolated signs, allowing more expressive and fluid communication.
8. **Flexibility Across Environments:** Maintains accuracy despite variations in lighting, user position, and background, increasing usability in everyday settings.

3.4 Feasibility Study

The feasibility study evaluates whether the proposed system is practical and beneficial for implementation.

3.4.1 Technical Feasibility

The proposed system is technically feasible because it uses commonly available hardware and software tools. A standard webcam is sufficient to capture hand gestures. The system is implemented using Python programming language and Media-Pipe framework, which are open-source and widely supported.

The processing requirements are moderate and can run on a normal laptop or desktop computer without the need for high-end hardware. Therefore, the system can be easily developed and deployed.

3.4.2 Economic Feasibility

The proposed system is economically feasible because it does not require expensive hardware components. It uses:

- A standard webcam
- A personal computer or laptop
- Open-source software tools

Since no sensor gloves or specialized devices are required, the overall development cost is low. This makes the system affordable and suitable for educational and assistive applications.

3.4.3 Operational Feasibility

The system is operationally feasible as it is simple to use and requires minimal training. Users only need to perform predefined hand gestures in front of the camera.

The speech output feature ensures smooth communication in real-world environments such as classrooms, hospitals, and offices.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system architecture of the Sign Language to Speech Converter consists of four main components: image acquisition, hand tracking, gesture recognition, and speech generation.

Initially, the webcam captures live video frames of the user's hand gestures. These frames are processed using Media-Pipe to detect hand landmarks. The detected landmarks are then analysed to recognize predefined gestures. Once the gesture is identified, the system converts it into text and generates speech output using Text-to-Speech (TTS) technology.The overall architecture ensures real-time processing and smooth communication between the user and the listener.

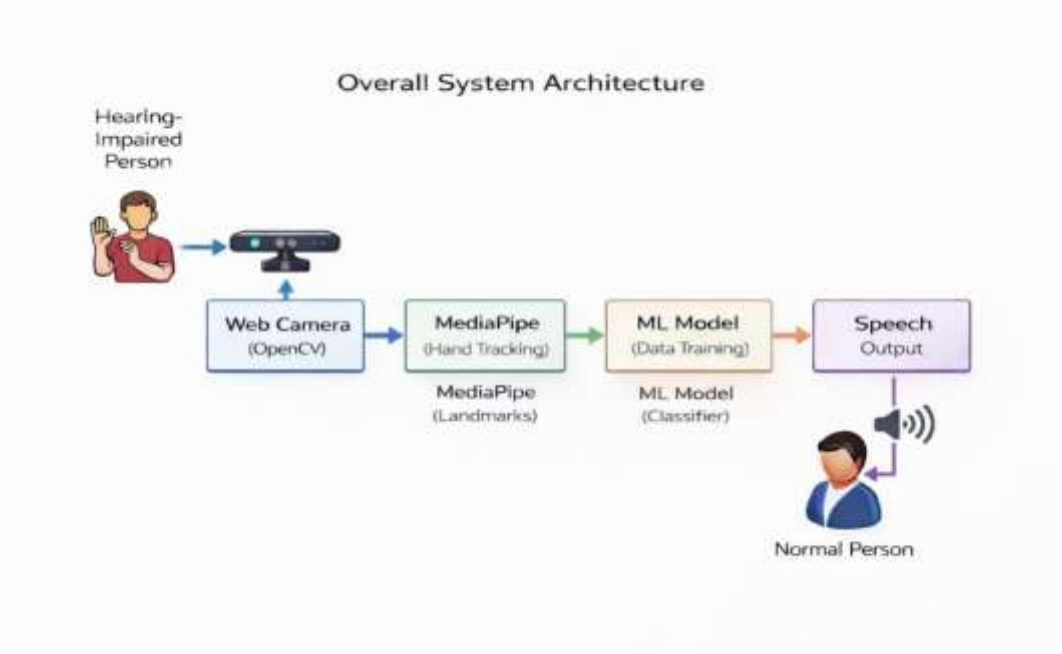


Fig 4.1 System Architecture

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) represents the flow of data within the Sign Language to Speech Converter system. It shows how input data is processed and transformed into output.

4.2.1 DFD Level 0 (Context Diagram)

The Level 0 DFD provides a high-level view of the entire system.

In this system:

• The User performs hand gestures.
• The gesture is given as input to the system through a webcam.
• The system processes the gesture.
• The system generates speech output.

Flow Description:

1. The User provides hand gesture as input.
2. The system captures and processes the gesture.
3. The system returns speech output to the User.

This level shows the overall interaction between the user and the system without internal module details.

4.2.2 DFD Level 1

The Level 1 DFD explains the internal processing steps of the system in detail.

The main processes involved are:

1. **Image Capture Process**
   The webcam captures live video frames of the hand gesture.
2. **Hand Landmark Detection Process**
   The captured frame is processed using Media-Pipe to detect hand landmarks.

3. **Feature Extraction Process**

   The system extracts x and y coordinate values of the detected landmarks.

4. **Gesture Recognition Process**

   The extracted features are compared with predefined gesture patterns stored in the system database.

5. **Text Conversion Process**

   Once the gesture is recognized, it is converted into corresponding text.

6. **Speech Generation Process**

   The text is converted into audio output using Text-to-Speech (TTS).

4.2.3 Data Stores and Data Flow

The system uses internal storage to help recognize gestures and generate speech. The gesture database stores predefined hand gestures and their labels, while feature storage temporarily keeps the hand landmark points for processing. The audio storage holds the generated speech before it is played to the user.

The flow of data starts with the webcam capturing hand movements, which are then processed by the hand detection module. The detected hand points are sent to the feature extraction module, where they are converted into feature vectors. These features are compared with the gesture database to recognize the gesture. Once the gesture is recognized, it is converted into text and then transformed into speech using the speech generation module. Finally, the generated speech is output to the user. This shows how the system effectively transforms hand gestures into meaningful audio output.

Data Flow Diagram

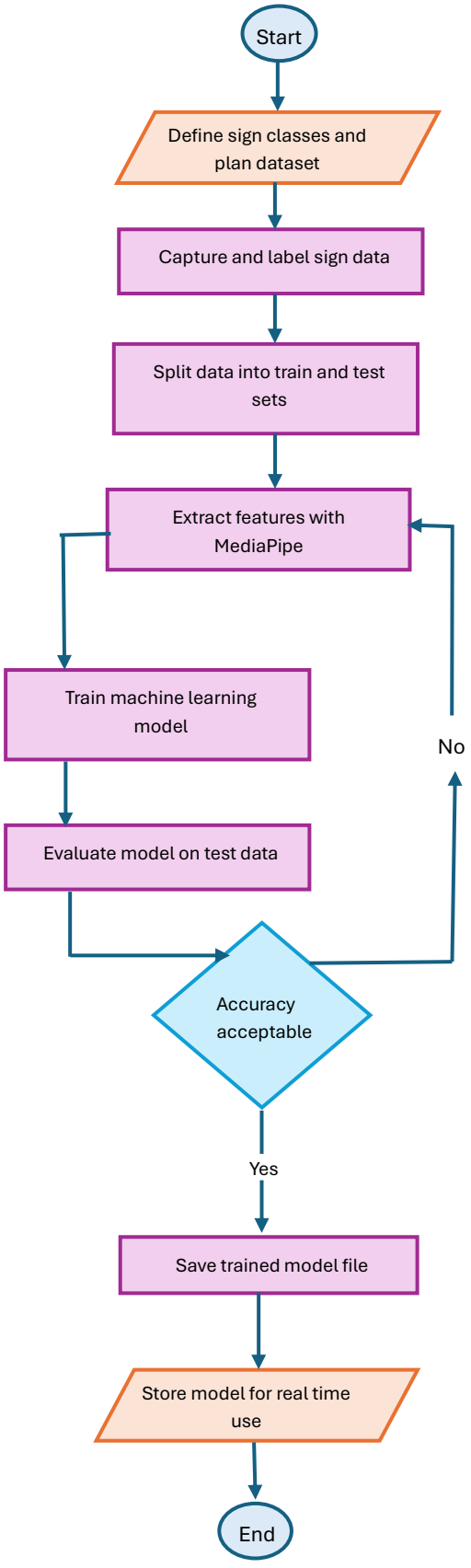


Fig 4.2.3 Data Flow Diagram

4.3 Use Case Diagram

The Use Case Diagram illustrates interactions between the user and the system.

Primary Actors:

User – Performs gestures in front of the camera.

System – Recognizes gestures and outputs speech.

Use Cases:

Capture Gesture – The system captures hand movements from the live feed.

Process Gesture – The system analyses and extracts gesture features.

Recognize Gesture – The system classifies the gesture.

Generate Speech – The system converts recognized gestures into audible speech.

This provides a clear understanding of functional requirements and user interactions.

4.4 Module Description

The system consists of four main modules that work together to enable accurate sign language recognition and communication. The Hand Tracking Module captures real-time hand movements and identifies hand positions using computer vision techniques, providing sequential data for analysis. The Feature Extraction Module processes this data to extract important features such as finger angles, joint positions, and palm orientation, converting them into numerical representations suitable for classification. The Gesture Classification Module uses machine learning algorithms, including CNN, SVM, or LSTM, to recognize gestures and generate corresponding labels, improving accuracy over time with new data. Finally, the Speech Generation Module converts recognized gestures into text and natural-sounding speech using TTS engines like Google Text-to-Speech, enabling clear communication for users unfamiliar with sign language.

4.5 Sign to Speech Data Flow Diagram

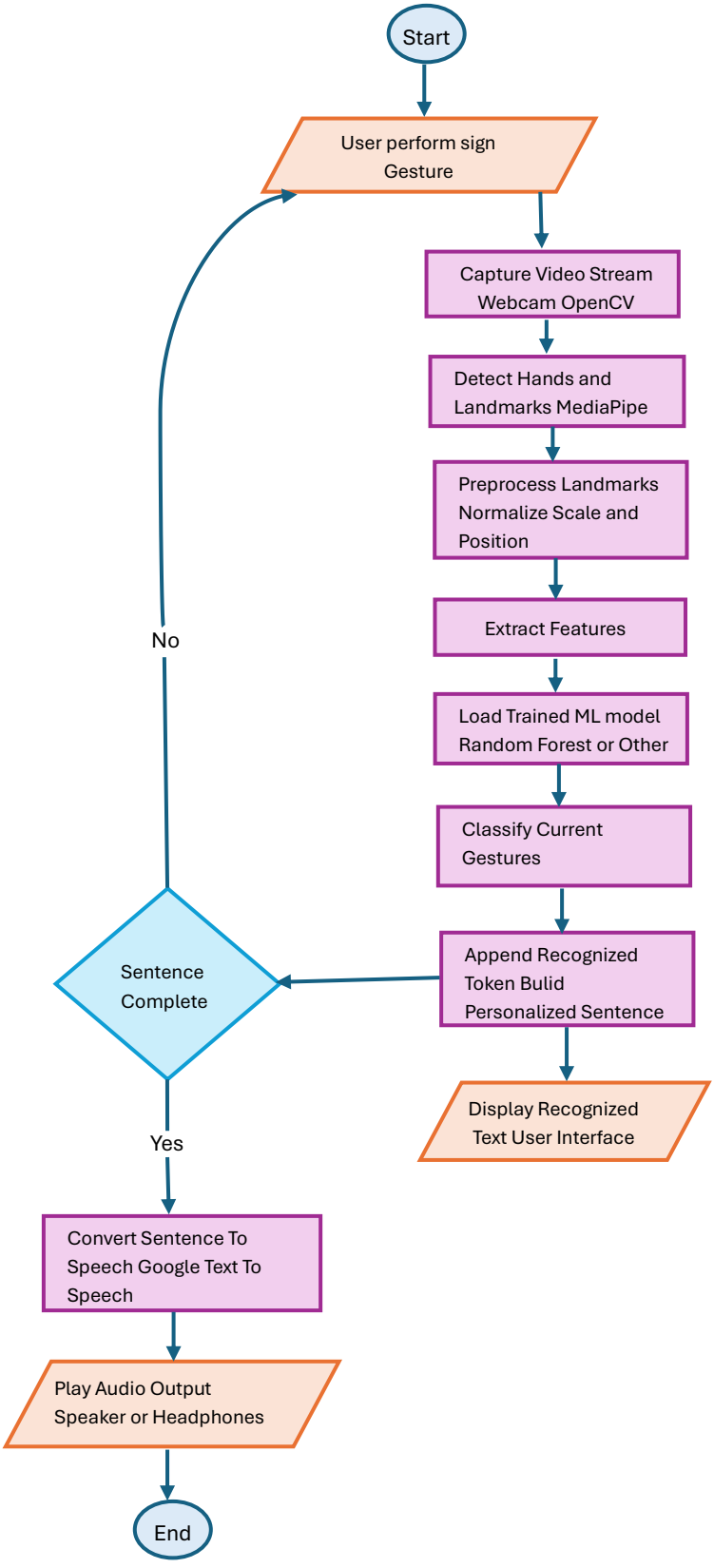


Fig 4.5 sign to speech Data Flow Diagram

CHAPTER 5

IMPLEMENTATION

5.1 Tools and Technologies Used

The system was developed using a set of carefully selected programming languages, libraries, and development tools to ensure efficient hand tracking, accurate gesture recognition, and seamless integration of all modules. The following tools and technologies were employed:

- **Python** – Primary programming language for algorithm development, computer vision, and system implementation.
- **OpenCV** – Used for hand detection, tracking, and image preprocessing.
- **Media-Pipe** – Provides ready-to-use frameworks for real-time hand tracking and feature extraction.
- **Google Text-to-Speech (GTTS)** – Converts recognized gestures into natural-sounding speech.
- **Visual Studio Code (VS Code)** – Development environments for coding, testing, and debugging.
- **Git & GitHub** – Version control tools for code management, collaboration, and reproducibility.

5.2 Dataset Preparation

A dataset of hand gestures was collected to train and validate the recognition system. The dataset consisted of images and videos of various gestures captured under different lighting conditions and backgrounds to enhance the system's robustness. Each gesture was carefully labeled and pre-processed to standardize hand positions, orientations, and sizes, ensuring consistency across the dataset and improving the accuracy of gesture recognition.

Multiple Subjects – Gestures were performed by multiple individuals with different hand sizes and skin tones to ensure the system can generalize effectively across diverse users.

Variety of Gestures – The dataset includes a wide range of commonly used gestures to cover all intended system interactions and improve recognition performance.

Consistent Preprocessing – All images and videos were resized, normalized, and formatted uniformly to maintain consistency and reduce computational errors during training.

SIGN GESTURES – SIGN 1



Fig 5.2.1 “How are you” Sign

SIGN 2









Fig 5.2.2 “I need water” Sign

SIGN 3









Fig 5.2.3 “Hello” Sign

SIGN 4









Fig 5.2.4 “I need help” Sign

SIGN 5









Fig 5.2.5 “I am hungry” Sign

SIGN 6









Fig 5.2.6 “No” Sign

5.3 Model Training

The gesture recognition system was trained using traditional machine learning algorithms. Features extracted from hand images, such as finger positions, angles, and relative distances, were used as input to the classifier. The dataset was divided into training and testing sets to evaluate model performance and ensure generalization. Hyperparameters were carefully tuned to optimize the model's accuracy, and the system was validated using metrics such as accuracy, precision, and recall. This approach allowed the model to effectively recognize gestures while maintaining computational efficiency suitable for real-time implementation.

5.4 Software Requirements

1. **Python**: The primary programming language for implementation due to its extensive libraries and ease of use.
2. **Media-Pipe**: For real-time hand and finger tracking and gesture recognition.
3. **OpenCV**: For image and video processing, including capturing frames from the camera.
4. **Machine Learning Libraries**: Such as TensorFlow or PyTorch, used to build and train the sign recognition models.
5. **Speech Synthesis Library**: Like Pyttsx3 or gTTS, to convert recognized sign sentences into spoken output.
6. **IDE or Code Editor**: Examples include Python IDLE, Visual Studio Code, or Jupyter Notebook for development and debugging.

Operating System: A compatible OS such as Windows or Linux to run the software stack.

5.4 Hardware Requirements

1. **Camera**: A standard webcam or any high-resolution camera capable of capturing clear hand gestures in real-time.
2. **Computer/Laptop**: A device with sufficient processing power (preferably with at least a quad-core CPU and 8GB RAM) to run Media-Pipe, machine learning models, and OpenCV efficiently.
3. **GPU (Optional)**: A graphics processing unit can accelerate model training and real-time inference but is not mandatory.
4. **Speakers**: For outputting the converted speech from recognized sign language.
5. **Stable Lighting Environment**: Although the system can work under various backgrounds, adequate lighting helps improve gesture detection accuracy.

5.5 System Integration

After training, the model was integrated with the other system modules. The hand tracking, feature extraction, gesture classification, and speech generation modules were connected to work in real-time. The system continuously captures hand gestures, processes them, recognizes the gesture, and outputs the corresponding text and speech, enabling seamless interaction with users.

CHAPTER 6

TRAINING OF RANDOM FOREST CLASSIFIER

6.1 Overview

This document outlines the training process and evaluation metrics for a Random Forest Classifier used in the Sign Recognition project. The model is trained on a dataset of 5,989 samples with 42 features per sample, categorizing inputs into 6 distinct classes.

ASPECT	DETAILS
Purpose	Train an ML model for sign language/gesture recognition
Architecture	Random Forest Classifier (Ensemble Method)
Features per Sample	42 numerical features
Target Classes	6 (Classes '0' through '5')
Total Samples	5,989

Table 6.1 – Random Forest Model Specifications

6.2 Data Distribution & Splitting

The dataset exhibits a highly balanced distribution across all 6 classes, ensuring that the model does not become biased toward any majority class.

CLASS	SAMPLES	PERCENTAGE
Class 0	996	16.6%
Class 1	1000	16.7%
Class 2	1000	16.7%
Class 3	1000	16.7%
Class 4	993	16.6%
Class 5	1000	16.7%
Total	**5989**	**100%**

Table 6.2 – Dataset Class Distribution

Data Splitting Strategy (60/20/20)

- **Training Set:** 3593 samples (60%) – Used for model fitting.
- **Validation Set:** 1198 samples (20%) – Used for hyperparameters and intermediate evaluation.
- **Test Set:** 1198 samples (20%) – Used for final, unbiased evaluation metrics.

6.3 Training & Validation Process

The Random Forest Classifier was fitted using the training data segment. Through cross-validation (evaluating the model across different data folds), we achieved excellent and stable performance.

Overall Training & Evaluation Metrics

The following summary encapsulates the fundamental performance characteristics of the model:

METRIC	VALUE	EXPLANATION
Total Samples	5989	The absolute size of our entire dataset across all 6 classes combined.
Training Samples	3593	Represents the 60% partition of the dataset that the model explicitly learned from.
Validation Accuracy	99.92%	The performance during intermediate checks on 20% of the dataset, assisting tuning.
Test Accuracy	99.92%	The final evaluation metric on the completely hidden test partition (remaining 20%).
Cross-Val Accuracy	99.80% (±0.23%)	Proves model stability across diverse folds; low variance shows consistent performance.

Table 6.3 – Summary of Model Performance Metrics

6.4 Evaluation on Unseen Test Data

The trained model was evaluated against the unseen test dataset (1,198 samples). It achieved an exceptional final Test Accuracy of 99.92%.

6.4.1 Classification Report

The metrics show essentially perfect precision, recall, and f1-score across all 6 classes.

CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT
0	1.00	1.00	1.00	199
1	1.00	1.00	1.00	200
2	1.00	1.00	1.00	200
3	1.00	1.00	1.00	200
4	1.00	0.99	1.00	199
5	1.00	1.00	1.00	200
Accuracy			**1.00**	**1198**
Macro Avg	**1.00**	**1.00**	**1.00**	**1198**
Weighted Avg	**1.00**	**1.00**	**1.00**	**1198**

Table 6.4.1 – Classification Performance across All Classes

6.4.2 Confusion Matrix

The confusion matrix has been rendered below as a heatmap to highlight the predictive accuracy visually. Only a single misclassification occurred across the entire test set (one sample of Class 4 was predicted as Class 1).

True \ Predicted	0	1	2	3	4	5
0	199	0	0	0	0	0
1	0	200	0	0	0	0
2	0	0	200	0	0	0
3	0	0	0	200	0	0
4	0	1	0	0	198	0
5	0	0	0	0	0	200

Table 6.4.2 – Heatmap Representation of Confusion Matrix

APPENDIX – I

CODING

Collect image.py

```
# Import necessary libraries
import os   # For interacting with the operating system
import cv2   # For accessing the webcam and image processing
import mediapipe as mp   # For hand detection
import pickle   # For saving dataset

# Set up MediaPipe for hand tracking
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

# Create hands detector (same settings as working test)
hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)
# Define the data directory
DATA_DIR = 'data'
PICKLE_FILE = 'data.pickle'

# Create the data directory if it does not exist
if not os.path.exists(DATA_DIR):
    os.makedirs(DATA_DIR)
```

```
# Define the number of classes and dataset size
number_of_classes = 6# Total number of classes for data collection
dataset_size = 1000  # Maximum number of images to capture per class

# Initialize the webcam feed
cap = cv2.VideoCapture(0)

# Current class index
current_class = 0
counter = 0
is_capturing = False
hand_detected = False

# Dataset storage (landmark data)
all_data = []
all_labels = []

# Create directory for current class
def ensure_class_dir(class_idx):
    class_dir = os.path.join(DATA_DIR, str(class_idx))
    if not os.path.exists(class_dir):
        os.makedirs(class_dir)
    return class_dir

# Get current image count for a class
def get_image_count(class_idx):
    class_dir = os.path.join(DATA_DIR, str(class_idx))
    if os.path.exists(class_dir):
        return len([f for f in os.listdir(class_dir) if f.endswith('.jpg')])
```

```
    return 0

# Reset (delete all images) for a class
def reset_class(class_idx):
    class_dir = os.path.join(DATA_DIR, str(class_idx))
    if os.path.exists(class_dir):
        for f in os.listdir(class_dir):
            if f.endswith('.jpg'):
                os.remove(os.path.join(class_dir, f))
    return 0

# Extract normalized landmark data
def extract_landmarks(hand_landmarks):
    x_ = []
    y_ = []
    data_aux = []

    # Get all coordinates
    for landmark in hand_landmarks.landmark:
        x_.append(landmark.x)
        y_.append(landmark.y)

    # Normalize coordinates
    for landmark in hand_landmarks.landmark:
        data_aux.append(landmark.x - min(x_))
        data_aux.append(landmark.y - min(y_))
    return data_aux

# Save dataset to pickle
```

```
def save_dataset():
    with open(PICKLE_FILE, 'wb') as f:
        pickle.dump({'data': all_data, 'labels': all_labels}, f)


# Display instructions
def draw_ui(frame, current_class, counter, is_capturing, hand_detected,
total_samples):
    h, w = frame.shape[:2]


    # Draw semi-transparent overlay for instructions
    overlay = frame.copy()
    cv2.rectangle(overlay, (10, 10), (450, 220), (0, 0, 0), -1)
    cv2.addWeighted(overlay, 0.6, frame, 0.4, 0, frame)


    # Title
    cv2.putText(frame, 'Sign Language Data Collection', (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)


    # Hand detection status
    if hand_detected:
        cv2.putText(frame, 'HAND DETECTED', (250, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
    else:
        cv2.putText(frame, 'NO HAND', (280, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
    # Current status
    status = "CAPTURING" if is_capturing else "PAUSED"
    status_color = (0, 255, 0) if is_capturing else (0, 165, 255)
```

```
    cv2.putText(frame, f'Class: {current_class} | Images: {counter} | {status}',
(20, 70),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, status_color, 2)

    # Dataset info
    cv2.putText(frame, f'Total Dataset Samples: {total_samples}', (20, 95),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)

    # Controls
    cv2.putText(frame, 'Controls:', (20, 120),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
    cv2.putText(frame, 'S - Start/Stop | C - Capture Single', (20, 140),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (200, 200, 200), 1)
    cv2.putText(frame, 'N - Next Class | P - Previous Class', (20, 160),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (200, 200, 200), 1)
    cv2.putText(frame, 'R - Reset Class | ESC - Save & Quit', (20, 180),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (200, 200, 200), 1)
    cv2.putText(frame, '(Saves images + landmarks in real-time)', (20, 200),
                cv2.FONT_HERSHEY_SIMPLEX, 0.4, (150, 150, 150), 1)

    # Progress bar
    progress = min(counter / dataset_size, 1.0)
    bar_width = 300
    cv2.rectangle(frame, (20, h - 40), (20 + bar_width, h - 20), (50, 50, 50), -1)
    cv2.rectangle(frame, (20, h - 40), (20 + int(bar_width * progress), h - 20), (0,
255, 0), -1)
    cv2.putText(frame, f'{counter}/{dataset_size}', (330, h - 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
```

```
    return frame

# Ensure initial class directory exists
ensure_class_dir(current_class)
counter = get_image_count(current_class)

print("=" * 50)
print("Sign Language Data Collection")
print("(Saves images + landmarks in real-time!)")
print("=" * 50)
print("Controls:")
print("  S     - Start/Stop continuous capture")
print("  C     - Capture single image")
print("  N     - Next class")
print("  P     - Previous class")
print("  R     - Reset class (delete all images)")
print("  ESC   - Save dataset & Quit")
print("=" * 50)

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: Could not read from camera")
        break
    # Flip frame horizontally for mirror effect
    frame = cv2.flip(frame, 1)

    # Convert to RGB for MediaPipe
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
    # Detect hands
    results = hands.process(rgb_frame)
    hand_detected = results.multi_hand_landmarks is not None

    # Store current landmarks if detected
    current_landmarks = None
    if hand_detected:
        current_landmarks = extract_landmarks(results.multi_hand_landmarks[0])

    # Create display frame
    display_frame = frame.copy()

    # Draw hand landmarks if detected
    if hand_detected:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                display_frame,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS,
                mp_drawing_styles.get_default_hand_landmarks_style(),
                mp_drawing_styles.get_default_hand_connections_style()

    # Draw UI
    display_frame = draw_ui(display_frame, current_class, counter, is_capturing,
hand_detected, len(all_data))

    # Show the frame
    cv2.imshow('Sign Language Data Collection', display_frame)
```

```
    # Continuous capture mode - ONLY SAVE IF HAND IS DETECTED
    if is_capturing and counter < dataset_size and hand_detected and current_landmarks:
        class_dir = ensure_class_dir(current_class)
        img_path = os.path.join(class_dir, f'{counter}.jpg')
        cv2.imwrite(img_path, frame)  # Save original frame without landmarks

        # Save landmark data directly!
        all_data.append(current_landmarks)
        all_labels.append(str(current_class))

        counter += 1
        print(f'Captured: Class {current_class}, Image {counter}, Landmarks saved!')

    # Handle key presses
    key = cv2.waitKey(10) & 0xFF

    if key == 27:  # ESC - Save and Quit
        print("\nSaving dataset and exiting...")
        save_dataset()
        break
    elif key == ord('s') or key == ord('S'):  # S - Start/Stop capture
        is_capturing = not is_capturing
        if is_capturing:
            print(f"\n>>> Started capturing for Class {current_class} (waiting for hand...)")
        else:
```

```
            print(f"\n>>> Paused capturing. Class {current_class}: {counter} images")

    elif key == ord('c') or key == ord('C'):  # C - Capture single image
        if hand_detected and current_landmarks:
            if counter < dataset_size:
                class_dir = ensure_class_dir(current_class)
                img_path = os.path.join(class_dir, f'{counter}.jpg')
                cv2.imwrite(img_path, frame)

                # Save landmark data directly!
                all_data.append(current_landmarks)
                all_labels.append(str(current_class))

                counter += 1
                print(f'Captured: Class {current_class}, Image {counter}, Landmarks saved!')
            else:
                print(f"Class {current_class} already has {dataset_size} images!")
        else:
            print("Cannot capture - No hand detected!")

    elif key == ord('n') or key == ord('N'):  # N - Next class
        is_capturing = False
        current_class = (current_class + 1) % number_of_classes
        ensure_class_dir(current_class)
        counter = get_image_count(current_class)
        print(f"\n>>> Switched to Class {current_class} ({counter} existing images)")
```

```
    elif key == ord('p') or key == ord('P'):  # P - Previous class
        is_capturing = False
        current_class = (current_class - 1) % number_of_classes
        ensure_class_dir(current_class)
        counter = get_image_count(current_class)
        print(f"\n>>> Switched to Class {current_class} ({counter} existing
images)")


    elif key == ord('r') or key == ord('R'):  # R - Reset class
        is_capturing = False
        counter = reset_class(current_class)
        # Also remove landmarks for this class from dataset
        all_data = [d for d, l in zip(all_data, all_labels) if l != str(current_class)]
        all_labels = [l for l in all_labels if l != str(current_class)]
        print(f"\n>>> Reset Class {current_class} - All images and landmarks
deleted!")

# Release resources
hands.close()
cap.release()
cv2.destroyAllWindows()

# Final save
save_dataset()

print("\n" + "=" * 50)
print("Data collection complete!")
print("=" * 50)
print("Images Summary:")
```

```
for i in range(number_of_classes):
    count = get_image_count(i)
    print(f"  Class {i}: {count} images")
print(f"\nDataset saved to: {PICKLE_FILE}")
print(f"Total samples: {len(all_data)}")
print("=" * 50)
```

Create DataSet.py

```
# Import necessary libraries
import os  # For interacting with the operating system
import pickle  # For serializing and de-serializing Python objects
import mediapipe as mp  # For utilizing the MediaPipe library for hand tracking
import cv2  # For image processing

# Set up MediaPipe for hand tracking
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

# Use EXACT same settings as collect_imgs.py (video mode works better)
hands = mp_hands.Hands(
    static_image_mode=False,  # Same as collect_imgs.py
    max_num_hands=1,
    min_detection_confidence=0.5,  # Same as collect_imgs.py
    min_tracking_confidence=0.5   # Same as collect_imgs.py
)

# Define the data directory
DATA_DIR = 'data/'

# Initialize lists to store data and corresponding labels
data = []
labels = []

# Counters for statistics
total_images = 0
```

```
detected_images = 0
failed_images = 0

print("=" * 50)
print("Creating Dataset from Hand Images")
print("Press any key to skip image, ESC to quit")
print("=" * 50)

# Loop through the directories in the data directory
for dir_ in sorted(os.listdir(DATA_DIR)):
    dir_path = os.path.join(DATA_DIR, dir_)
    if not os.path.isdir(dir_path):
        continue

    print(f"\nProcessing Class {dir_}...")
    class_detected = 0
    class_failed = 0

    # Loop through image files in the current directory
    image_files = sorted([f for f in os.listdir(dir_path) if f.endswith(('.jpg', '.jpeg',
'.png'))])

    # WARM-UP: Process first few images multiple times to initialize tracking
    warmup_count = min(5, len(image_files))
    print(f"  Warming up detector with first {warmup_count} images...")
    for i in range(warmup_count):
        img = cv2.imread(os.path.join(dir_path, image_files[i]))
        if img is not None:
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
        # Process multiple times for warm-up
        for _ in range(3):
            hands.process(img_rgb)
print("  Warm-up complete!")

for idx, img_path in enumerate(image_files):
    total_images += 1
    full_path = os.path.join(dir_path, img_path)

     # Read the image
    img = cv2.imread(full_path)
    if img is None:
        print(f"  Warning: Could not read {img_path}")
        failed_images += 1
        class_failed += 1
        continue

    # Flip image (since we flipped during capture)
    # img = cv2.flip(img, 1)  # Try uncommenting if detection fails

    # Convert BGR to RGB
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Process the image - try multiple times if needed
    results = None
    for attempt in range(3):  # Try up to 3 times
        results = hands.process(img_rgb)
        if results.multi_hand_landmarks:
            break
```

```
# Create display image
 display_img = img.copy()

# If hand landmarks are detected
if results.multi_hand_landmarks:
    # Initialize lists for storing x and y coordinates
    x_ = []
    y_ = []

    # Loop through each detected hand (take only the first one)
    for hand_landmarks in results.multi_hand_landmarks:
        # Draw landmarks on display image
        mp_drawing.draw_landmarks(
            display_img,
            hand_landmarks,
            mp_hands.HAND_CONNECTIONS,
            mp_drawing_styles.get_default_hand_landmarks_style(),
            mp_drawing_styles.get_default_hand_connections_style()
        )

        # Extract and store the x and y coordinates of each landmark
        for i in range(len(hand_landmarks.landmark)):
            x = hand_landmarks.landmark[i].x
            y = hand_landmarks.landmark[i].y
            x_.append(x)
            y_.append(y)

        # Normalize the data and append it to the data_aux list
```

```
            data_aux = []
            for i in range(len(hand_landmarks.landmark)):
                x = hand_landmarks.landmark[i].x
                y = hand_landmarks.landmark[i].y
                data_aux.append(x - min(x_))
                data_aux.append(y - min(y_))

            # Append the normalized data to the main data list and the label to the
labels list
            data.append(data_aux)
            labels.append(dir_)
            detected_images += 1
            class_detected += 1

            # Show success status
            cv2.putText(display_img, f'DETECTED - Class {dir_}', (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
            break  # Only process first hand
        else:
            failed_images += 1
            class_failed += 1
            # Show fail status
            cv2.putText(display_img, f'NO HAND - Class {dir_}', (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

        # Show progress
        cv2.putText(display_img, f'Image {idx+1}/{len(image_files)}', (10, 60),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
```

```
        cv2.putText(display_img, f'Detected: {class_detected} | Failed:
{class_failed}', (10, 85),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

        # Display the image
        cv2.imshow('Processing Images', display_img)

        # Wait for key (1ms) - increase to see images longer
         key = cv2.waitKey(1) & 0xFF
        if key == 27:  # ESC to quit
            print("\nQuitting early...")
            break

    print(f"  Class {dir_}: {class_detected} detected, {class_failed} failed")

    # Check if ESC was pressed
    if cv2.waitKey(1) & 0xFF == 27:
        break

cv2.destroyAllWindows()

# Save the data and labels as a dictionary in a pickle file
f = open('data.pickle', 'wb')
pickle.dump({'data': data, 'labels': labels}, f)
f.close()

print("\n" + "=" * 50)
print("Dataset Creation Complete!")
print("=" * 50)
```

```
print(f"Total images processed: {total_images}")
print(f"Hands detected: {detected_images}")
print(f"Detection failed: {failed_images}")
print(f"Detection rate: {detected_images/total_images*100:.1f}%" if total_images > 0 else "N/A")
print(f"\nSaved to: data.pickle")
print("=" * 50)
```

Train Classifier.py

```
# Import necessary libraries
import pickle  # For serializing and de-serializing Python objects
from sklearn.ensemble import RandomForestClassifier  # For creating a Random Forest Classifier model
from sklearn.model_selection import train_test_split, cross_val_score  # For splitting data and cross-validation
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  # For evaluation
import numpy as np  # For numerical operations
import matplotlib.pyplot as plt  # For plotting
import seaborn as sns  # For better visualizations

print("=" * 60)
print("Sign Language Recognition - Model Training")
print("=" * 60)

# Load the data from 'data.pickle'
print("\n[1/5] Loading dataset...")
data_dict = pickle.load(open('data.pickle', 'rb'))

# Convert data and labels to NumPy arrays
data = np.asarray(data_dict['data'])
labels = np.asarray(data_dict['labels'])

print(f"   Total samples: {len(data)}")
print(f"   Features per sample: {len(data[0]) if len(data) > 0 else 0}")
print(f"   Classes: {np.unique(labels)}")

# Show class distribution
```

```
print("\n[2/5] Class distribution:")
unique_labels = np.unique(labels)
class_counts = []
for label in unique_labels:
    count = np.sum(labels == label)
    class_counts.append(count)
    print(f"      Class {label}: {count} samples ({count/len(labels)*100:.1f}%)")

# Split the data into training and testing sets
print("\n[3/5] Splitting data (60% train, 20% validation, 20% test)...")
x_temp, x_test, y_temp, y_test = train_test_split(
    data, labels, test_size=0.2, shuffle=True, stratify=labels, random_state=42
)
x_train, x_val, y_train, y_val = train_test_split(
    x_temp, y_temp, test_size=0.25, shuffle=True, stratify=y_temp,
random_state=42
)
print(f"      Training set: {len(x_train)} samples")
print(f"      Validation set: {len(x_val)} samples")
print(f"      Test set: {len(x_test)} samples")

# Initialize an optimized Random Forest Classifier model
print("\n[4/5] Training Random Forest Classifier...")
model = RandomForestClassifier(
    n_estimators=100,       # Number of trees
    max_depth=20,           # Maximum depth of trees
    min_samples_split=5,    # Minimum samples to split
    min_samples_leaf=2,     # Minimum samples in leaf
    random_state=42,        # For reproducibility
```

```
    n_jobs=-1              # Use all CPU cores
)

# Train the model using the training data
model.fit(x_train, y_train)
print("      Training complete!")

# Validate the model
print("\n[5/5] Evaluating model performance...")

# Validation accuracy
val_predict = model.predict(x_val)
val_score = accuracy_score(y_val, val_predict)
print(f"\n      Validation Accuracy: {val_score * 100:.2f}%")

# Test accuracy
y_predict = model.predict(x_test)
test_score = accuracy_score(y_test, y_predict)
print(f"      Test Accuracy: {test_score * 100:.2f}%")

# Cross-validation score
cv_scores = cross_val_score(model, data, labels, cv=5)
print(f"      Cross-Validation Accuracy: {cv_scores.mean() * 100:.2f}% (+/-
{cv_scores.std() * 100:.2f}%)")

# Classification Report
print("\n" + "-" * 60)
print("Classification Report (Test Set):")
print("-" * 60)
```

```
print(classification_report(y_test, y_predict))

# Confusion Matrix
cm = confusion_matrix(y_test, y_predict)
print("-" * 60)
print("Confusion Matrix:")
print("-" * 60)
print(cm)

# Save the trained model to a pickle file
print("\n" + "=" * 60)
model_file = 'model.p'
with open(model_file, 'wb') as f:
    pickle.dump({'model': model}, f)
print(f"Model saved to: {model_file}")

#
==========================================================
===
# PLOTTING SECTION
#
==========================================================
===
print("\nGenerating plots...")

# Set style
plt.style.use('seaborn-v0_8-darkgrid')

# Plot 1: Class Distribution
plt.figure(figsize=(8, 6))
```

```
colors = plt.cm.viridis(np.linspace(0.3, 0.7, len(unique_labels)))
bars = plt.bar(unique_labels, class_counts, color=colors, edgecolor='black')
plt.xlabel('Class')
plt.ylabel('Number of Samples')
plt.title('Class Distribution')
for bar, count in zip(bars, class_counts):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 10,
             str(count), ha='center', va='bottom', fontweight='bold')
plt.tight_layout()
plt.savefig('class_distribution.png', dpi=150, bbox_inches='tight')

# Plot 2: Train/Val/Test Split
plt.figure(figsize=(8, 6))
split_labels = ['Train', 'Validation', 'Test']
split_sizes = [len(x_train), len(x_val), len(x_test)]
pie_colors = ['#2ecc71', '#3498db', '#e74c3c']
wedges, texts, autotexts = plt.pie(split_sizes, labels=split_labels,
autopct='%1.1f%%',
                                   colors=pie_colors, explode=(0.05, 0.05, 0.05),
                                   shadow=True, startangle=90)
plt.title('Dataset Split')
plt.tight_layout()
plt.savefig('dataset_split.png', dpi=150, bbox_inches='tight')

# Plot 3: Confusion Matrix Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=unique_labels, yticklabels=unique_labels,
            cbar=True, annot_kws={'size': 14, 'weight': 'bold'})
```

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')

# Plot 4: Accuracy Comparison
plt.figure(figsize=(8, 6))
accuracy_labels = ['Validation', 'Test', 'Cross-Val']
accuracy_values = [val_score * 100, test_score * 100, cv_scores.mean() * 100]
bar_colors = ['#9b59b6', '#e74c3c', '#f39c12']
bars = plt.bar(accuracy_labels, accuracy_values, color=bar_colors,
edgecolor='black')
plt.ylim(0, 110)
plt.ylabel('Accuracy (%)')
plt.title('Model Accuracy')
plt.axhline(y=100, color='green', linestyle='--', alpha=0.5, label='Perfect')
for bar, val in zip(bars, accuracy_values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
             f'{val:.1f}%', ha='center', va='bottom', fontweight='bold')
plt.tight_layout()
plt.savefig('model_accuracy.png', dpi=150, bbox_inches='tight')

# Plot 5: Feature Importance (Top 20)
plt.figure(figsize=(8, 6))
feature_importance = model.feature_importances_
top_n = min(20, len(feature_importance))
indices = np.argsort(feature_importance)[-top_n:]
```

```
plt.barh(range(top_n), feature_importance[indices], color='teal',
edgecolor='black')
plt.yticks(range(top_n), [f'Feature {i}' for i in indices])
plt.xlabel('Importance')
plt.title(f'Top {top_n} Feature Importance')
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')

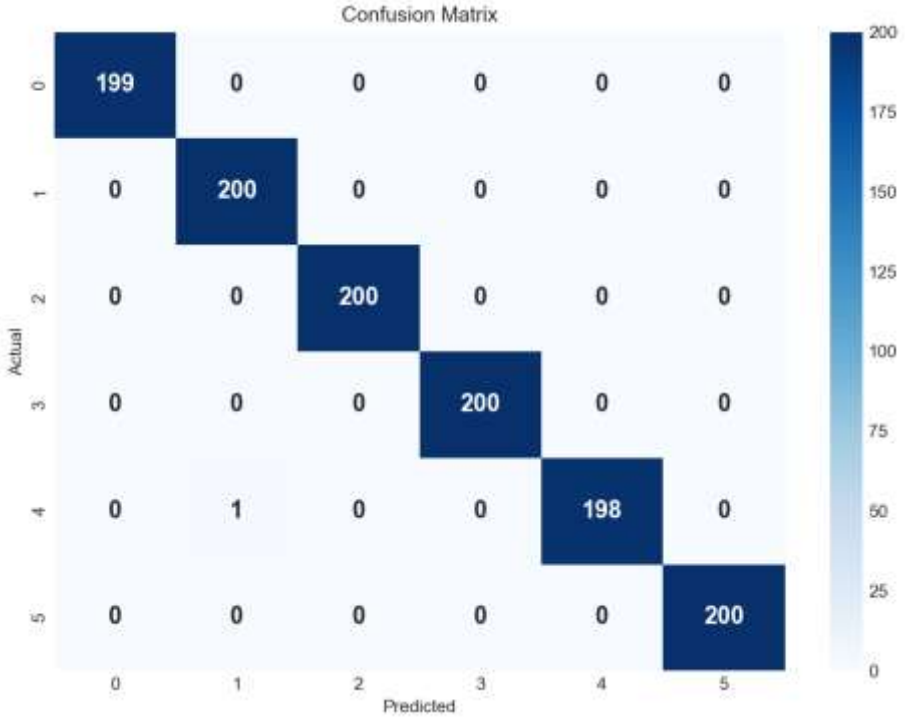
# Plot 6: Cross-Validation Scores
plt.figure(figsize=(8, 6))
fold_labels = [f'Fold {i+1}' for i in range(5)]
plt.bar(fold_labels, cv_scores * 100, color='#1abc9c', edgecolor='black')
plt.axhline(y=cv_scores.mean() * 100, color='red', linestyle='--', linewidth=2,
label=f'Mean: {cv_scores.mean()*100:.1f}%')
plt.ylim(0, 110)
plt.ylabel('Accuracy (%)')
plt.title('Cross-Validation Scores')
plt.legend()
for i, v in enumerate(cv_scores * 100):
    plt.text(i, v + 1, f'{v:.1f}%', ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig('cross_validation_scores.png', dpi=150, bbox_inches='tight')

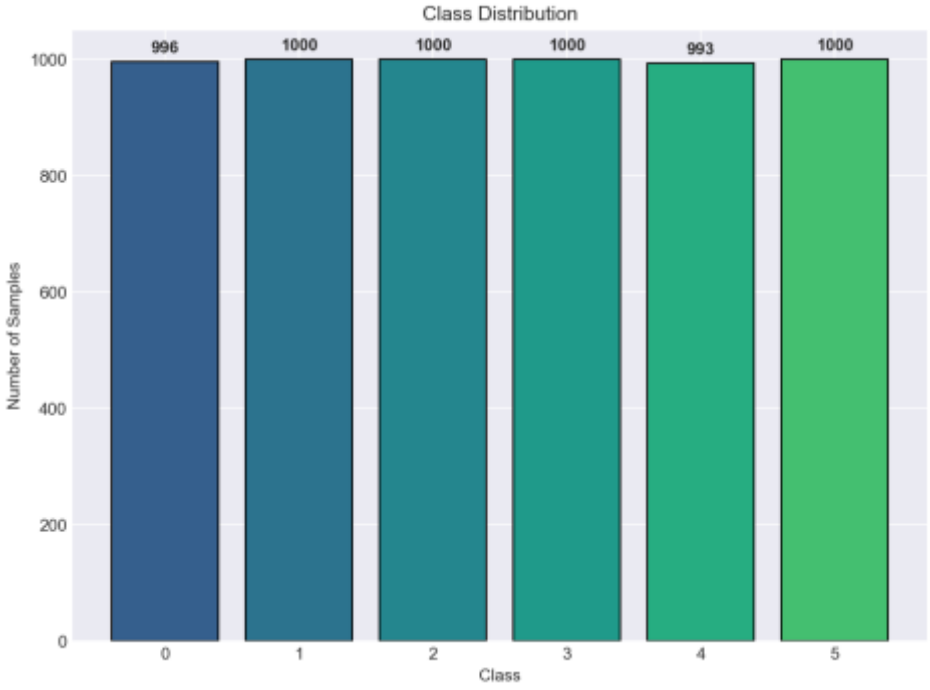
print("Plots saved separately as PNG files.")
plt.show()

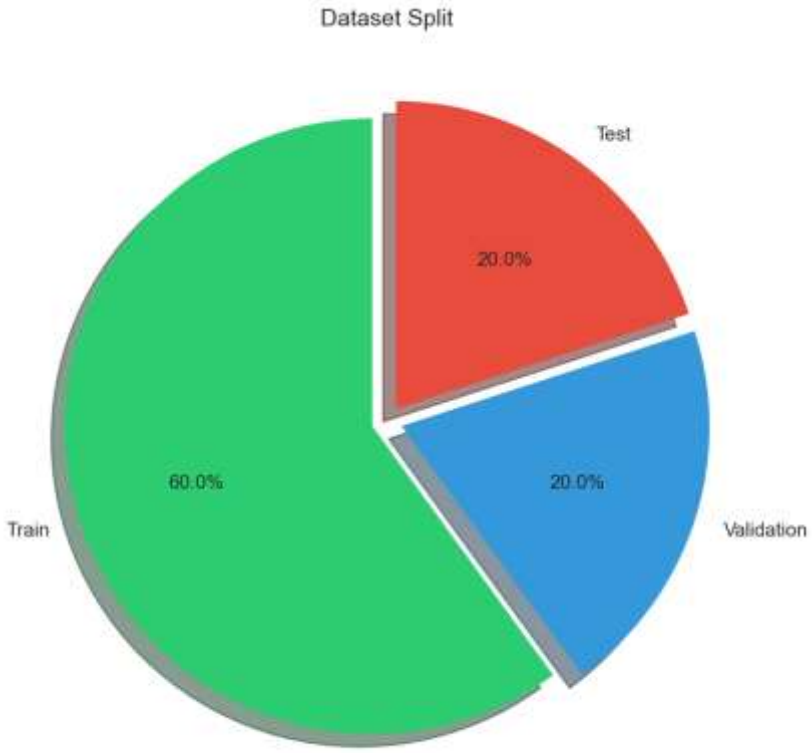
# Summary
print("\n" + "=" * 60)
print("TRAINING SUMMARY")
```

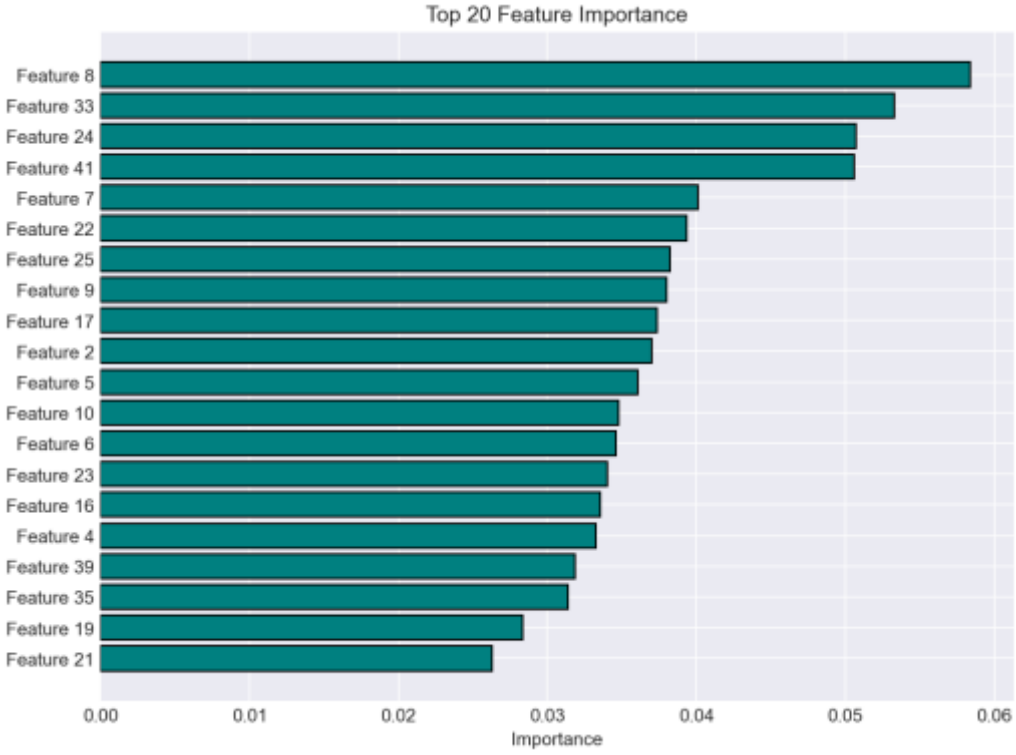
```
print("=" * 60)
print(f"  Total Samples:       {len(data)}")
print(f"  Training Samples:    {len(x_train)}")
print(f"  Validation Accuracy: {val_score * 100:.2f}%")
print(f"  Test Accuracy:       {test_score * 100:.2f}%")
print(f"  Cross-Val Accuracy:  {cv_scores.mean() * 100:.2f}%")
print(f"  Model File:          {model_file}")
print(f"  Plot Files:          Separate .png files generated")
print("=" * 60)
print("\nReady to use! Run Sign_classifier.py to test real-time recognition.")
```

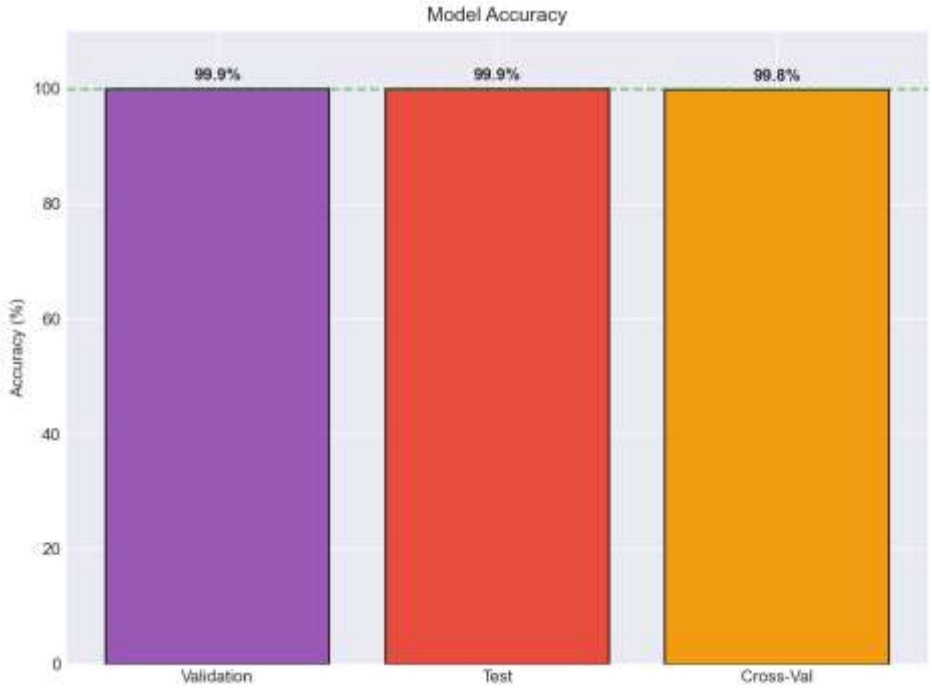
TRAINING RESULTS

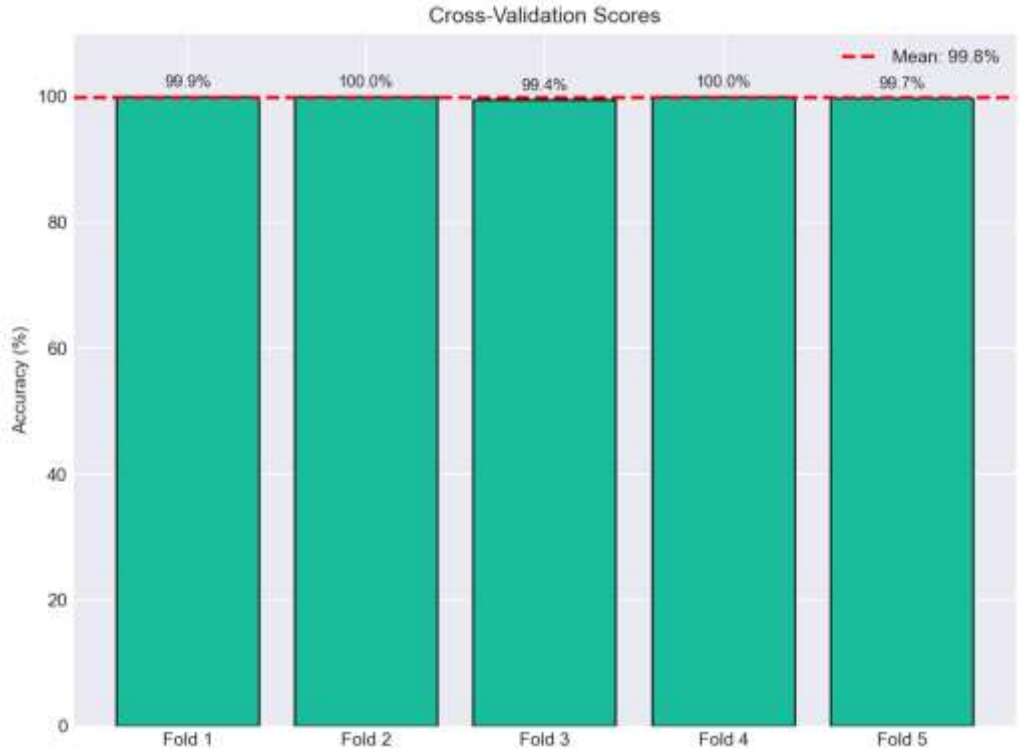





Dataset Split
Test
20.0%
60.0%
20.0%
Train
Validation


Top 20 Feature Importance
Feature 8
Feature 33
Feature 24
Feature 41
Feature 7
Feature 22
Feature 25
Feature 9
Feature 17
Feature 2
Feature 5
Feature 10
Feature 6
Feature 23
Feature 16
Feature 4
Feature 39
Feature 35
Feature 19
Feature 21
0.00
0.01
0.02
0.03
0.04
0.05
0.06
Importance


Model Accuracy
99.9%
99.9%
99.8%
100
80
60
40
20
0
Accuracy (%)
Validation
Test
Cross-Val


Cross-Validation Scores
Mean: 99.8%
99.9%
100.0%
99.4%
100.0%
99.7%
100
80
60
40
20
0
Accuracy (%)
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5

Sign Classifier.py

```
# Sign Language Classifier - Real-time Recognition
import pickle
import cv2
import mediapipe as mp
import numpy as np
from collections import deque
import time
import os
import threading
from gtts import gTTS
from audioplayer import AudioPlayer

# Audio settings
last_spoken_time = 0
audio_lock = threading.Lock()

def speak_text(text):
    """Convert text to speech and play it"""
    global last_spoken_time

    current_time = time.time()
    # Don't repeat too quickly (wait 3 seconds between same messages)
    if current_time - last_spoken_time < 3:
        return

    last_spoken_time = current_time

    def _play_audio():
```

```
        with audio_lock:
            try:
                # Create audio file
                tts = gTTS(text=text, lang='en')
                temp_file = "temp_voice.mp3"
                if os.path.exists(temp_file):
                    os.remove(temp_file)
                tts.save(temp_file)

                # Play audio
                AudioPlayer(temp_file).play(block=True)

                # Clean up
                if os.path.exists(temp_file):
                    os.remove(temp_file)
            except Exception as e:
                print(f"Audio Error: {e}")

    # Run in separate thread to not block video
    threading.Thread(target=_play_audio, daemon=True).start()

# Load the trained model
print("Loading model...")
model_dict = pickle.load(open('model.p', 'rb'))
model = model_dict['model']
print("Model loaded successfully!")

# Initialize webcam
cap = cv2.VideoCapture(0)
```

```
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# MediaPipe setup
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

# Class labels mapping
labels_dict = {
    '0': 'How are you',
    '1': 'Hello',
    '2': 'I need water',
    '3': 'I need help',
    '4': 'I am hungry',
    '5': 'No',

}

# Settings
MARGIN = 20
```

```
CONFIDENCE_THRESHOLD = 0.7  # Minimum confidence to display prediction
SMOOTHING_WINDOW = 5  # Number of frames to average for smoothing

# For prediction smoothing
prediction_history = deque(maxlen=SMOOTHING_WINDOW)
confidence_history = deque(maxlen=SMOOTHING_WINDOW)

# FPS calculation
fps_start_time = time.time()
fps_counter = 0
fps = 0

# Statistics
total_predictions = 0
class_counts = {label: 0 for label in labels_dict.values()}

def get_smoothed_prediction(predictions, confidences):
    """Get the most common prediction from recent history"""
    if not predictions:
        return None, 0

    # Count occurrences
    pred_counts = {}
    for pred in predictions:
        pred_counts[pred] = pred_counts.get(pred, 0) + 1

    # Get most common
    most_common = max(pred_counts, key=pred_counts.get)
```

```
    avg_confidence = np.mean([c for p, c in zip(predictions, confidences) if p ==
most_common])

    return most_common, avg_confidence

def draw_ui(frame, prediction, confidence, hand_detected, fps):
    """Draw the UI overlay"""
    h, w = frame.shape[:2]

    # Top bar background
    overlay = frame.copy()
    cv2.rectangle(overlay, (0, 0), (w, 80), (30, 30, 30), -1)
    cv2.addWeighted(overlay, 0.7, frame, 0.3, 0, frame)

    # Title
    cv2.putText(frame, 'Convey Message', (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)

    # FPS
    cv2.putText(frame, f'FPS: {fps:.1f}', (w - 100, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

    # Status
    if hand_detected:
        status_text = "Hand Detected"
        status_color = (0, 255, 0)
    else:
        status_text = "Show your hand"
        status_color = (0, 165, 255)
```

```
    cv2.putText(frame, status_text, (10, 60),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, status_color, 2)

    # Prediction display (bottom)
    if prediction and confidence > CONFIDENCE_THRESHOLD:
        # Large prediction box at bottom
        box_h = 80
        cv2.rectangle(overlay, (0, h - box_h), (w, h), (0, 100, 0), -1)
        cv2.addWeighted(overlay, 0.7, frame, 0.3, 0, frame)

        # Prediction text
        cv2.putText(frame, prediction, (w//2 - 80, h - 25),
                    cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 255, 255), 4)

        # Confidence bar
        bar_width = int((w - 40) * confidence)
        cv2.rectangle(frame, (20, h - box_h + 10), (w - 20, h - box_h + 25), (50,
50, 50), -1)
        cv2.rectangle(frame, (20, h - box_h + 10), (20 + bar_width, h - box_h +
25), (0, 255, 0), -1)
        cv2.putText(frame, f'{confidence*100:.0f}%', (w - 60, h - box_h + 22),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 255, 255), 1)

    # Controls hint
    cv2.putText(frame, 'ESC to quit', (w - 100, 60),
                cv2.FONT_HERSHEY_SIMPLEX, 0.4, (150, 150, 150), 1)

    return frame
```

```
print("\n" + "=" * 50)
print("Sign Language Recognition - Real-time")
print("=" * 50)
print("Controls:")
print("  ESC - Quit")
print("=" * 50)
print("\nShow your hand to the camera...")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: Could not read from camera")
        break

    # Flip for mirror effect
    frame = cv2.flip(frame, 1)
    H, W, _ = frame.shape

    # Convert to RGB
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Process frame
    results = hands.process(frame_rgb)

    hand_detected = results.multi_hand_landmarks is not None
    current_prediction = None
    current_confidence = 0

    if hand_detected:
```

```
for hand_landmarks in results.multi_hand_landmarks:
    # Draw landmarks
    mp_drawing.draw_landmarks(
        frame,
        hand_landmarks,
        mp_hands.HAND_CONNECTIONS,
        mp_drawing_styles.get_default_hand_landmarks_style(),
        mp_drawing_styles.get_default_hand_connections_style()
    )

    # Extract features
    x_ = []
    y_ = []
    data_aux = []

    for landmark in hand_landmarks.landmark:
        x_.append(landmark.x)
        y_.append(landmark.y)

    for landmark in hand_landmarks.landmark:
          data_aux.append(landmark.x - min(x_))
        data_aux.append(landmark.y - min(y_))

    # Check feature count
    if len(data_aux) != 42:
        continue

    # Make prediction with probability
    try:
```

```
            prediction = model.predict([np.asarray(data_aux)])[0]
            probabilities = model.predict_proba([np.asarray(data_aux)])[0]
            confidence = max(probabilities)

            # Add to history for smoothing
            prediction_history.append(prediction)
            confidence_history.append(confidence)

            # Get smoothed prediction
            smoothed_pred, smoothed_conf = get_smoothed_prediction(
                list(prediction_history), list(confidence_history)
            )

            if smoothed_pred is not None:
                current_prediction = labels_dict.get(str(smoothed_pred),
str(smoothed_pred))
                current_confidence = smoothed_conf

                # Update statistics
                if current_confidence > CONFIDENCE_THRESHOLD:
                    total_predictions += 1
                    if current_prediction in class_counts:
                        class_counts[current_prediction] += 1

                    # Trigger audio
                    speak_text(current_prediction)

            # Draw bounding box
            x1 = int(min(x_) * W) - MARGIN
```

```
            y1 = int(min(y_) * H) - MARGIN
            x2 = int(max(x_) * W) + MARGIN
            y2 = int(max(y_) * H) + MARGIN

            # Color based on confidence
            if current_confidence > CONFIDENCE_THRESHOLD:
                box_color = (0, 255, 0)  # Green
            else:
                box_color = (0, 165, 255)  # Orange

            cv2.rectangle(frame, (x1, y1), (x2, y2), box_color, 3)

        except Exception as e:
            print(f"Prediction error: {e}")
    else:
        # Clear history when no hand detected
        prediction_history.clear()
        confidence_history.clear()

    # Calculate FPS
    fps_counter += 1
    if time.time() - fps_start_time >= 1:
        fps = fps_counter / (time.time() - fps_start_time)
        fps_counter = 0
        fps_start_time = time.time()

    # Draw UI
    frame = draw_ui(frame, current_prediction, current_confidence,
hand_detected, fps)
```

```
    # Show frame
    cv2.imshow('Sign Language Recognition', frame)

    # Handle key press
    key = cv2.waitKey(1) & 0xFF
    if key == 27:  # ESC
        break

# Cleanup
hands.close()
cap.release()
cv2.destroyAllWindows()

# Print statistics
print("\n" + "=" * 50)
print("Session Statistics")
print("=" * 50)
print(f"Total predictions: {total_predictions}")
for label, count in class_counts.items():
    if total_predictions > 0:
        percentage = count / total_predictions * 100
        print(f"  {label}: {count} ({percentage:.1f}%)")
print("=" * 50)
```

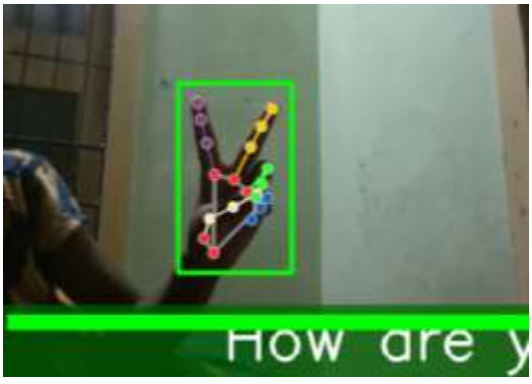
OUTPUTS

Output refers to the results generated by the system after processing the input data through various computational stages. In this project, the output includes the real-time visualization of hand gesture capture, accurate hand tracking and detection, and the corresponding speech generated from recognized gestures. These outputs validate the functionality, performance, and reliability of the system in achieving effective sign-to-speech communication.









The system accurately detects hand gestures in real time and converts them into both text and speech.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Model Performance Results

The Random Forest Classifier achieved high performance during the evaluation phase. The model obtained a cross-validation accuracy of 99.80% (+/- 0.23%), indicating consistent performance across different subsets of the dataset.

When tested on the 20% unseen test dataset (1,198 samples), the model maintained excellent performance. The classification report showed an F1-score of 1.00 for all six target classes. Out of the 1,198 test samples, 1,197 were correctly classified, while one instance from Class 4 was misclassified as Class 1.

7.2 Live Hand Capture

Shows the camera feed with the user performing a gesture.

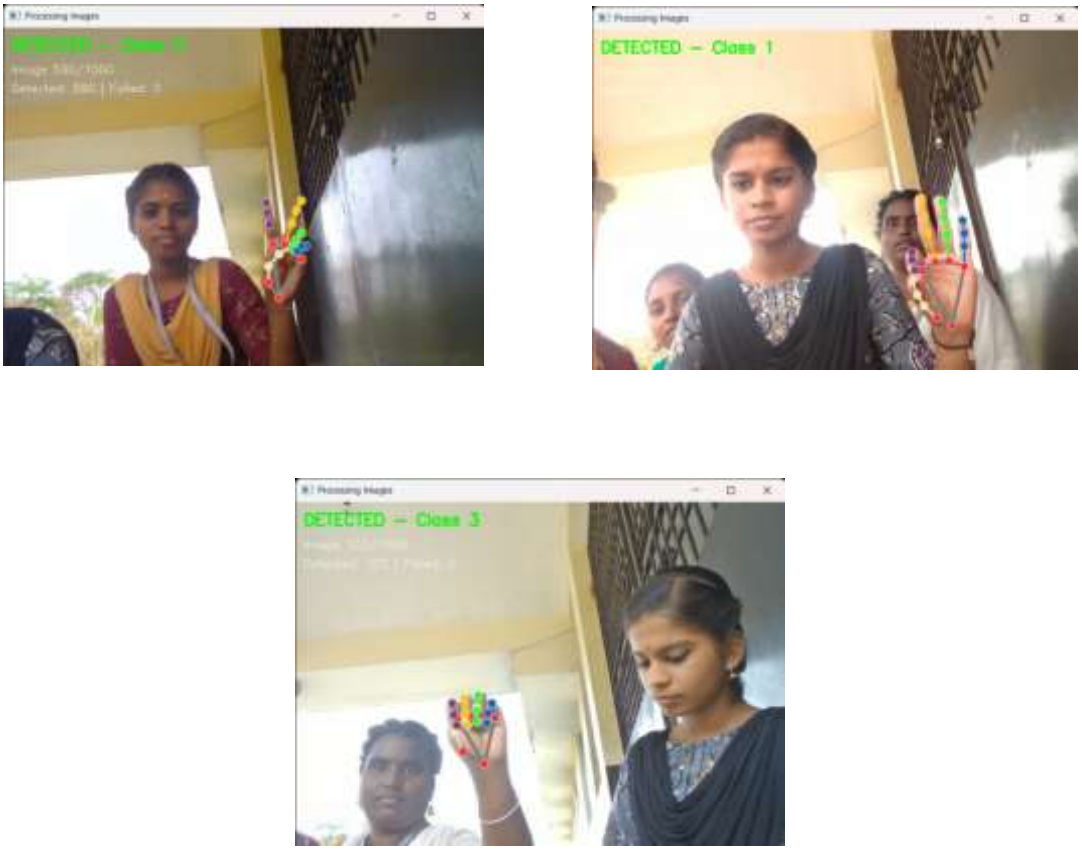
Live hand gesture capture refers to the process of using a webcam to continuously capture the user's hand movements in real time. The captured video frames are then used by the system to detect and process hand gestures for further recognition.



Fig 7.2 Live hand gesture capture

7.3 Hand Tracking & Detection

- Displays the hand with tracking markers or bounding boxes (from Media-Pipe).
- Demonstrates accurate detection of hand position and orientation.



Figs 7.3 Hand tracking using MediaPipe

7.4 Sign-to-Speech Output

This module demonstrates the audio output generated directly from the recognized hand gesture using the Google Text-to-Speech (gTTS) library. Once a gesture is detected, the system immediately produces the corresponding speech, enabling real-time communication for users unfamiliar with sign language.



Fig 7.4 Sign to Speech Convertor Output

7.5 Discussion

The high values of precision, recall, and F1-score indicate that the model performs highly accurate gesture classification. The near-perfect results demonstrate that the 42 extracted numerical features effectively represent the hand gestures.

Additionally, the dataset was balanced across all six classes (approximately 16.6% each), which helped reduce bias in the model and improved classification reliability. The minimal misclassification also suggests that the Random Forest algorithm is well-suited for gesture recognition tasks.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The hand gesture recognition system developed in this project successfully converts hand gestures into real-time speech, providing an effective communication solution for individuals unfamiliar with sign language. Using Media-Pipe for hand tracking and a machine learning-based classifier, the system accurately recognizes a variety of gestures and immediately generates audio feedback through the Sign-to-Speech module.

A diverse dataset of hand gestures was collected under varying lighting conditions, backgrounds, and hand orientations to ensure robustness. Preprocessing standardized hand positions and orientations, while the trained model achieved high accuracy, precision, and recall during testing and real-time experiments.

The system's modular architecture—including Hand Tracking, Feature Extraction, Gesture Classification, and Sign-to-Speech Output—ensures scalability and ease of integration for future improvements. The real-time performance demonstrates its practical applicability in educational, social, and assistive contexts, helping bridge the communication gap for users unfamiliar with sign language.

In summary, the project meets its objectives by combining computer vision, machine learning, and text-to-speech technologies to create a practical and efficient hand gesture recognition system. The results confirm that the system is accurate, robust, and user-friendly, providing a solid foundation for further development and enhancement.

8.2 Future Enhancements

While the current hand gesture recognition system demonstrates high accuracy and real-time performance, several improvements can make it more versatile, user-friendly, and widely applicable. One key enhancement is to expand the gesture vocabulary, allowing the system to recognize a wider range of signs and support more complex communication. This would involve collecting additional gesture data and refining the model to handle multiple classes efficiently.

Another important improvement is mobile and portable integration, enabling users to access the system on smartphones or tablets. Optimizing the system for devices with limited computational resources will make it practical for real-world, on-the-go use.

Additional enhancements include:

- Expanded Gesture Vocabulary – Incorporating additional hand gestures will allow users to communicate more complex messages, increasing the system's applicability.
- Mobile and Portable Integration – Adapting the system for smartphones or tablets will enable on-the-go use and broader accessibility.
- Advanced Machine Learning Models – Utilizing deep learning approaches, such as convolutional or recurrent neural networks, can improve recognition of subtle or complex gestures and increase overall accuracy.
- Multilingual Speech Output – Integrating support for multiple languages will make the system accessible to a wider range of users.
- Enhanced User Interface – Providing interactive visual feedback and gesture prompts will improve usability, particularly for children, elderly users, and individuals with disabilities.

- Integration with Virtual/Augmented Reality – The system can be extended to VR/AR platforms or connected with other assistive technologies, providing immersive and seamless gesture-based communication.

Collectively, these enhancements will make the system more comprehensive, robust, and practical, expanding its applications in educational, social, and assistive contexts while maintaining high performance and reliability.

Software and Libraries

1.Google Developers. MediaPipe Hands – On-device Real-time Hand Tracking. Available at: https://chuoling.github.io/mediapipe/solutions/hands.html

2.Google. gTTS (Google Text-to-Speech) Python Library Documentation. Available at: https://pypi.org/project/gTTS/

3.Python Software Foundation. Python Programming Language Documentation. Available at: https://www.python.org/doc/

4.OpenCV Documentation. Computer Vision Library for Python. Available at: https://opencv.org/

Datasets

5. Custom Hand Gesture Dataset. Collected under varying lighting conditions and backgrounds for real-time gesture recognition.

6. Koller, O., Zargaran, S., Ney, H., & Bowden, R. (2016). Deep Hand: How to Train a CNN on 1 Million Hand Images When Your Data is Continuous and Weakly Labelled. CVPR.

APPENDIX II

LIST OF REFERENCES

[1] “Hand Gesture Recognition Using MediaPipe and CNN for Indian Sign Language and Conversion to Speech Format for Indian Regional Languages,” *IEEE Xplore*, 2025.

[2] “Hand Landmark Distance Based Sign Language Recognition Using MediaPipe,” *IEEE Xplore*, 2024.

[3] “Real-Time Hand Gesture Recognition Using MediaPipe and Artificial Neural Networks,” *IEEE Conference Publication*, 2024.

[4] “Indian Sign Language Recognition Using LSTM and MediaPipe,” *IEEE Xplore*, 2025.

[5] “Women’s Safety Through Real-Time Gesture Recognition Using MediaPipe and Machine Learning Algorithms,” *IEEE Xplore*, 2025.

[6] C. Lugaresi et al., “MediaPipe: A Framework for Building Perception Pipelines,” *arXiv preprint*, 2019.

[7] L. N. Lavanya, H. R. Sujay, A. M. et al., “Real-Time Sign Language Recognition and Multilingual Speech Output Based on Machine Learning,” *International Journal of Human Computations & Intelligence*, 2026.

[8] R. A. A. Ilham et al., “Dynamic Sign Language Recognition Using MediaPipe Library and Modified LSTM Method,” *International Journal on Advanced Science, Engineering and Information Technology*, 2023.

[9] P. Lin, “Study on Real-Time Gesture Recognition Using Convolutional Neural Network Based on SMOTE Method,” *Academic Journal of Science and Technology*, 2026.

[10] Y. Meng, H. Jiang, N. Duan, and H. Wen, “Real-Time Hand Gesture Monitoring Model Based on MediaPipe’s Registerable System,” Sensors, vol. 24, no. 19, 6262, 2024.

[11] R. Awalkar et al., “Silent Expressions: Two-Handed Indian Sign Language Recognition Using MediaPipe and Machine Learning,” International Journal of Innovative Science and Research Technology, 2025.

[12] L. N. Lavanya et al., “Real-Time Sign Language Recognition and Multilingual Speech Output Based on Machine Learning,” International Journal of Human Computations and Intelligence, vol. 5, no. 1, pp. 656–663, 2025.

[13] R. S. Bhalerao et al., “Sign Language Recognition,” IJRASET – Journal for Research in Applied Science and Engineering Technology, 2025.

[14] K. K. Sukhadan et al., “Sign Language Recognition System,” IJRASET, 2024.

[15] “Real-Time Gesture Recognition for Sign Language Using Machine Learning,” IEEE Conference Publication, 2024.

[16] “Gesture Recognition for Sign Language Using Machine Learning,” IEEE Conference Publication, 2024.

[17] Python Software Foundation, Python Language Reference. Available: https://www.python.org/

[18] OpenCV Library, Open Source Computer Vision Library. Available: https://opencv.org/

[19] gTTS Developers, Google Text-to-Speech (gTTS) Library. Available: https://pypi.org/project/gTTS/

[20] MediaPipe Developers, MediaPipe Hands – On-device Real-time Hand Tracking. Available: https://chuoling.github.io/mediapipe/solutions/hands.html

APPENDIX III



T.J.S. ENGINEERING COLLEGE

Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai,
Accredited by NAAC / ISO 9001:2015 Certified Institution.

An Autonomous Institution

NCRTSET-2026

CERTIFICATE OF PARTICIPATION

This is to certify that Dr/ Mr / Mrs/ Ms. YAMUNA T
from ARTIFICIAL INTELLIGENCE AND DATA SCIENCE has participated and presented
paper on REAL TIME PERSONALISED SIGN
LANGUAGE TO SPEECH CONVERTOR
In the National Conference on Recent Trends in Science, Engineering & Technology
(NCRTSET-2026), held on 11th April 2026.

CONVENOR/HOD-ECE PRINCIPAL DIRECTOR



T.J.S. ENGINEERING COLLEGE

Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai.
Accredited by NAAC : ISO 9001:2015 Certified Institution.

An Autonomous Institution

NCRTSET-2026

CERTIFICATE OF PARTICIPATION

This is to certify that Dr/ Mr / Mrs/ Ms. MONISHA. A
from ARTIFICIAL INTELLIGENCE AND DATA SCIENCE has participated and presented paper on REAL TIME PERSONALISED SIGN LANGUAGE TO SPEECH CONVERTOR

In the National Conference on Recent Trends in Science , Engineering & Technology (NCRTSET-2026) , held on 11th April 2026.

CONVENOR/HOD-ECE | PRINCIPAL | DIRECTOR



T.J.S. ENGINEERING COLLEGE

Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai,
Accredited by NAAC / ISO 9001:2015 Certified Institution.

An Autonomous Institution

NCRTSET-2026

CERTIFICATE OF PARTICIPATION

This is to certify that Dr/ Mr / Mrs/ Ms. KUMUDHAVALLI. S
from ARTIFICIAL INTELLIGENCE AND DATA SCIENCE has participated and presented
paper on REAL TIME PERSONALISED SIGN
LANGUAGE TO SPEECH CONVERTOR
In the National Conference on Recent Trends in Science, Engineering & Technology
(NCRTSET-2026), held on 11th April 2026.

CONVENOR/HOD-ECE PRINCIPAL DIRECTOR



T.J.S. ENGINEERING COLLEGE

Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai,
Accredited by NAAC / ISO 9001:2015 Certified Institution.

An Autonomous Institution

NCRTSET-2026

CERTIFICATE OF PARTICIPATION

This is to certify that Dr/ Mr / Mrs/ Ms. NAVEENA. B
from ARTIFICIAL INTELLIGENCE AND DATA SCIENCE has participated and presented
paper on REAL TIME PERSONALISED SIGN
LANGUAGE TO SPEECH CONVERTOR.
In the National Conference on Recent Trends in Science , Engineering & Technology
(NCRTSET-2026) , held on 11th April 2026.

CONVENOR/HOD-ECE PRINCIPAL DIRECTOR